OOP Probetestat 2 – Coding

Kapitel 1–4 · Übungsblätter OOP01–OOP04 · C++17 · pbma · Google Test

Aufgabe	Situation	Zeit	Stoff	Dateien im Ordner
1	Messwerte aufbereiten	15 min	Referenzen, const-Referenzen, Rückgabe per Referenz, vector/erase	`test_messwerte.cpp`, `test_zusatz_messwerte.cpp`
2	Hitzewelle	20 min	pbma, Kommandozeile, Dateien, Formatierung, Exceptions	`juli.dat`, `kuehl.dat`, `kaputt.dat`, `august.dat`
3	Fahrradverleih	25 min	Klasse aus Text, const-Attribut, static, friend, nicht kopierbar	`test_fahrrad.cpp`, `test_zusatz_station.cpp`
4	Ringpuffer	30 min	Freispeicher, const-Überladung, Dreierregel, Copy & Swap	`test_ringpuffer.cpp`, `test_zusatz_ringpuffer.cpp`

Die `test_zusatz_*.cpp` gehören zu den Teilen b) und kommen erst ins Projekt, wenn Sie diese bearbeiten. Lösungen: `loesungen/` – erst nach dem Durchlauf öffnen.

Bearbeitungszeit: **90 Minuten** für die Kernaufgaben 1–4 (jeweils Teil a). Die Teile b) sind Transfer und Variation. Bearbeiten Sie sie erst, wenn alle Kernaufgaben fertig sind, oder später als Übung.

Hilfsmittel: Skript, Übungsblätter, cppreference. Keine KI.

Hinweise

- Jede Aufgabe hat ihren Ordner mit Tests und Daten. Legen Sie pro Aufgabe ein eigenes Projekt an und kopieren Sie die Dateien aus dem Aufgabenordner hinein.
- Die Hilfsdateien `alltest.h`, `gtest_main.cpp`, `pbma.h` und `pbma.cpp` liegen in `vorlagen/`. Kopieren Sie sie wie auf den Übungsblättern in jedes Projekt, das sie braucht. Für Unit-Tests: Linker mit `-pthread` und `-lgtest`, keine eigene `main`.
- Datei- und Klassennamen müssen exakt so heißen, wie es die Tests verlangen.
- Übersetzt wird mit `-std=c++17 -Wall -Wextra -pedantic`. Code mit Warnungen gilt als unsauber. Speicherfehler und Lecks führen zu Abzug; prüfen Sie mit valgrind.

Aufgabe 1 – Messwerte aufbereiten (15 min)

Ordner: `aufgabe1_messwerte/`, Test: `test_messwerte.cpp`

Ein Temperatursensor liefert seine Messreihen als `std::vector<int>` in Zehntelgrad. Bevor die Werte weiterverarbeitet werden, braucht die Auswertung eine kleine Bibliothek `messwerte` (Header und Implementierung) mit diesen Funktionen:

- **begrenze:** Bringt alle Werte einer Messreihe in einen erlaubten Bereich [min, max]. Werte darunter werden auf min gesetzt, Werte darüber auf max. Zurückgegeben wird, wie viele Werte geändert wurden. Ist der Bereich unsinnig, wird mit `std::runtime_error("invalid range")` abgebrochen, ohne dass sich die Messreihe ändert.

- **groesster:** Liefert den größten Wert so, dass der Aufrufer ihn direkt in der Messreihe korrigieren kann. Gibt es mehrere gleich große, gilt der erste. Bei einer leeren Messreihe: `std::runtime_error("empty")`.
- **kennzahlen:** Berechnet für eine Messreihe Mittelwert und Spannweite (Maximum − Minimum) und gibt beide an den Aufrufer zurück. Die Messreihe wird dabei weder kopiert noch verändert. Leere Messreihe: `std::runtime_error`.
- **entferne_ausreisser:** Entfernt alle Werte, deren Betrag größer als eine Grenze ist. Die Reihenfolge der übrigen bleibt erhalten. Zurückgegeben wird die Anzahl der entfernten Werte. Negative Grenze: `std::runtime_error`.

Die genauen Prototypen leiten Sie aus dem Text und dem Test ab.

b) Transfer (Test: `test_zusatz_messwerte.cpp`)

- **glaette:** Erzeugt aus einer Messreihe eine **neue** Messreihe mit gleitendem Mittelwert über ein Fenster von k Werten (ganzzahlig, abgeschnitten). Aus n Werten werden n − k + 1 Werte. Fenster 0 oder größer als die Messreihe: `std::runtime_error`.
- **normiere:** Dieselbe Bibliothek soll auch alte C-Felder (`int`-Feld plus Länge) verarbeiten können. Sie verschiebt alle Werte so, dass das Minimum 0 wird, und gibt die Verschiebung an den Aufrufer zurück. Leeres Feld oder `nullptr`: `std::runtime_error("empty")`.

Aufgabe 2 – Hitzewelle (20 min)

Ordner: `aufgabe2_hitze/`, Programm: `hitze.cpp`, Daten: `juli.dat`, `kuehl.dat`, `kaputt.dat`, `august.dat`

Ein Wetterdienst liefert Tageshöchsttemperaturen in Zehntelgrad. Die erste Zahl der Datei gibt die Anzahl der folgenden Tage an (wie `zahlen.dat` auf Blatt 2). Schreiben Sie ein Programm `hitze`, das eine solche Datei auswertet. Verwenden Sie `pbma` für Einlesen und Kommandozeile.

- Erstes Positionsargument ist der Dateiname, Vorgabe `juli.dat`.
- Ein Tag ist ein Hitzetag, wenn er mindestens die Grenze erreicht. Die Grenze ist standardmäßig 30,0 °C und lässt sich mit `-grenze=N` (in Zehntelgrad) ändern.
- Stimmt die Anzahl in der Datei nicht oder enthält die Datei keine Tage, bricht das Programm mit einer `std::runtime_error` ab (Text frei, aber aussagekräftig).
- Mit `-v` wird zuerst jeder Tag ausgegeben, Hitzetage mit einem Stern markiert (Format siehe unten).
- Danach folgen immer: Anzahl der Tage, Anzahl der Hitzetage, die längste zusammenhängende Hitzeperiode mit ihrem ersten Tag (bei Gleichstand die frühere) und der Mittelwert in °C auf zwei Nachkommastellen.
- Temperaturen werden in °C mit genau einer Nachkommastelle ausgegeben.

Erwartete Ausgaben, die exakt so stimmen müssen:

```
$ ./hitze
Tage: 31
Hitzetage (>= 30.0): 16
Laengste Hitzeperiode: 6 Tage ab Tag 23
Mittel: 29.35

$ ./hitze juli.dat -grenze=310
Tage: 31
Hitzetage (>= 31.0): 8
Laengste Hitzeperiode: 4 Tage ab Tag 8
Mittel: 29.35

$ ./hitze kuehl.dat -v
Tag  1:  -1.2
Tag  2:   3.5
Tag  3:  10.8
Tag  4:   9.9
Tage: 4
Hitzetage (>= 30.0): 0
Laengste Hitzeperiode: 0 Tage
Mittel: 5.75
```

Bei `./hitze juli.dat -v` sehen die ersten Zeilen so aus:

```
Tag  1:  24.1
Tag  2:  25.6
Tag  3:  26.8
Tag  4:  28.4
Tag  5:  30.1 *
Tag  6:  30.7 *
Tag  7:  29.9
```

`./hitze kaputt.dat` muss abbrechen. Mit `august.dat` wird der Gleichstand bei der längsten Hitzeperiode geprüft:

```
$ ./hitze august.dat
Tage: 10
Hitzetage (>= 30.0): 6
Laengste Hitzeperiode: 2 Tage ab Tag 1
Mittel: 26.81
```

b) Variation

- Mit dem Flag `-frost` erscheint vor dem Mittelwert zusätzlich die Zeile `Frosttage: n` (Tage unter 0,0 °C).
- Mit der Option `-o=datei` landet die Zusammenfassung (alles ab `Tage:`) nicht auf der Konsole, sondern in dieser Datei. Die `-v`-Liste bleibt auf der Konsole. Eine nicht schreibbare Datei führt zum Abbruch.
- Schreiben Sie den Ausgabecode dabei nur **einmal**, nicht für Konsole und Datei getrennt.

Aufgabe 3 – Fahrradverleih (25 min)

Ordner: `aufgabe3_verleih/`, Test: `test_fahrrad.cpp`

Ein städtischer Fahrradverleih möchte seine Räder verwalten. Schreiben Sie eine Klasse `Fahrrad` (`Fahrrad.h`, `Fahrrad.cpp`).

- Jedes Rad hat eine Rahmennummer, die sich nie ändert. Die Nummern werden automatisch fortlaufend ab 1001 vergeben und sind lückenlos: Ein fehlgeschlagener Versuch, ein Rad anzulegen, verbraucht keine Nummer.

- Beim Anlegen sind das Modell (Text) und optional der bisherige Kilometerstand anzugeben (Vorgabe 0). Ein negativer Kilometerstand bricht mit `std::runtime_error("negative km")` ab. Ein neues Rad gilt als frisch gewartet.
- Rahmennummer, Modell, Kilometerstand und ob das Rad gerade ausgeliehen ist, sind lesbar – auch bei Rädern, die nicht verändert werden dürfen. Das Modell wird dabei nicht kopiert.
- Ein Rad kann ausgeliehen werden, wenn es nicht schon unterwegs ist. Bei der Rückgabe werden die gefahrenen Kilometer angegeben. Eine Rückgabe ist nur möglich, wenn das Rad unterwegs ist und die Kilometer nicht negativ sind. Beide Aktionen melden zurück, ob sie geklappt haben. Bei einem Fehlschlag ändert sich nichts.
- Ein Rad braucht eine Wartung, wenn seit der letzten Wartung mindestens 500 km gefahren wurden; die Grenze kann bei der Abfrage auch anders angegeben werden. Warten ist nicht möglich, während das Rad unterwegs ist.
- Der Verleih will jederzeit wissen, wie viele Räder gerade unterwegs sind, ohne ein bestimmtes Rad zu kennen. Ein Rad, das zerstört wird, während es unterwegs ist (Diebstahl), zählt nicht mehr als unterwegs.
- Die Werkstatt kann die Modelle zweier Räder vertauschen (Umbau). Das geht nur, wenn beide in der Werkstatt stehen, also nicht ausgeliehen sind, und es zwei verschiedene Räder sind. Die Werkstatt-Funktion ist **keine** Methode, greift aber direkt auf die Daten der Räder zu.
- Ein Rad darf weder kopiert noch zugewiesen werden.

b) Transfer (Test: `test_zusatz_station.cpp`)

Der Verleih hat Stationen mit einer festen Anzahl Stellplätze. Schreiben Sie eine Klasse `Station` (`Station.h`, `Station.cpp`).

- Eine Station hat einen Namen und eine beim Anlegen festgelegte Zahl von Plätzen. 0 Plätze: `std::runtime_error("invalid size")`.
- Die Stellplätze verwalten Sie selbst in einem dynamisch angelegten Feld. Kein `std::vector`.
- Ein Rad kann eingestellt werden, wenn ein Platz frei ist, das Rad nicht unterwegs ist und nicht schon in dieser Station steht. Ein Rad kann über seine Rahmennummer wieder entnommen werden. Der Aufrufer bekommt das Rad zurück, oder „nichts“, wenn es nicht da ist.
- Abfragen: belegte und freie Plätze, ein Rad über seine Nummer finden (bei einer veränderbaren Station veränderbar, sonst nur lesend), Anzahl der Räder mit Wartungsbedarf (Grenze wie oben, Vorgabe 500).
- Eine Station **besitzt** ihre Räder nicht: Die Räder leben unabhängig von ihr. Eine Kopie einer Station hat ihre eigene Platzbelegung, verweist aber auf dieselben Räder.

Aufgabe 4 – Ringpuffer für Messwerte (30 min)

Ordner: `aufgabe4_ringpuffer/`, Test: `test_ringpuffer.cpp`

Ein Mikrocontroller soll immer nur die letzten N Messwerte im Speicher halten. Schreiben Sie eine Klasse `Ringpuffer` (`Ringpuffer.h`, `Ringpuffer.cpp`), die `int`-Werte in einem selbst verwalteten Feld im Freispeicher hält. Kein `std::vector`.

- Die Kapazität wird beim Anlegen angegeben. Eine Zahl darf nicht unbemerkt zu einem Ringpuffer werden. Kapazität 0: `std::runtime_error("invalid capacity")`.
- Neue Werte werden hinten angefügt. Ist der Puffer voll, wird der älteste Wert überschrieben. Das Anfügen lässt sich verketten: `r.add(1).add(2);`.
- Abfragen: aktuelle Anzahl, Kapazität, leer, voll.
- Zugriff über einen Index, wobei 0 der älteste noch vorhandene Wert ist. Bei veränderbaren Puffern kann der Wert so direkt geändert werden, bei konstanten nur gelesen. Ungültiger Index: `std::runtime_error("index out of range")`.
- Mittelwert aller vorhandenen Werte; bei leerem Puffer `std::runtime_error("empty")`.
- Der Puffer kann geleert werden; die Kapazität bleibt.

- Kopien und Zuweisungen sind vollständig unabhängig. Auch Selbst- und Kettenzuweisung müssen funktionieren. Kein Speicherleck, kein doppeltes Freigeben.

Hinweis: Überlegen Sie vor dem Programmieren auf Papier, welche Attribute Sie brauchen, um ohne Verschieben aller Werte auszukommen.

b) Variation (Test: `test_zusatz_ringpuffer.cpp`)

- Der neueste Wert ist direkt abfragbar; leer: `std::runtime_error("empty")`.
- Die Kapazität lässt sich nachträglich ändern. Dabei bleiben die neuesten Werte erhalten, so viele wie hineinpassen, mit dem ältesten wieder an Index 0. Neue Kapazität 0 wirft `"invalid capacity"`, und der Puffer bleibt unverändert.